

PARADIGMA ORIENTADO A OBJETOS

UNIDAD N° 2

Comportamiento de Objetos



SEMANA 4

Consideraciones previas

El contenido que se expone a continuación está ligado a los siguientes objetivos:

- Distingue el concepto de polimorfismo como principio dentro de la programación orientada a objetos.
- Distingue instrucciones para la lectura de teclado.
- Utiliza colecciones de datos para administrar información almacenada temporalmente.

Sobre las fuentes utilizadas en el material

El presente Material de Estudio constituye un ejercicio de recopilación de distintas fuentes, cuyas referencias bibliográficas estarán debidamente señaladas al final del documento. Este material, en ningún caso pretende asumir como propia la autoría de las ideas planteadas. La información que se incorpora tiene como única finalidad el apoyo para el desarrollo de los contenidos de la unidad correspondiente, respetando los derechos de autor ligados a las ideas e información seleccionada para los fines específicos de cada asignatura.

Introducción

Esta es la cuarta semana de la asignatura, y hasta esta altura se han visto varios temas de la programación orientada a objetos. En esta ocasión se analizará otro pilar de este paradigma, el polimorfismo. También se revisarán ejemplos prácticos sobre lectura de datos por medio del teclado.

En base a lo anterior, se retomará el concepto de almacenamiento de datos, en específico sobre el uso de colecciones. Para ello, se realizarán ejemplos en donde se crearán listas basadas en diferentes tipos de datos, se insertarán datos en ellas, se desplegarán los datos, se actualizarán y se eliminarán.

Ideas fuerza

1. **Polimorfismo:** es la capacidad que tienen ciertos lenguajes para hacer que, al enviar el mismo mensaje (o, en otras palabras, invocar al mismo método) desde distintos objetos, cada uno de esos objetos pueda responder a ese mensaje (o a esa invocación) de forma distinta.
2. **Scanner:** dentro del paquete `java.util`, `Scanner` es una clase que nos permite obtener la entrada de datos primitivos. Esto quiere decir que es posible capturar datos del tipo `int`, `double`, `string` y etc.

Índice

<i>1.- Polimorfismo</i>	<i>6</i>
1.1.- Ejemplo de Polimorfismo	6
1.2.- Polimorfismo paramétrico.....	8
1.3.- Polimorfismo de inclusión	10
<i>2.- Lectura de teclado.....</i>	<i>11</i>
2.1.- La clase Scanner	11
<i>3.- Administración de datos en colecciones</i>	<i>13</i>
3.2.- Acceder a un ítem (listar)	15
3.3.- Listar todos.....	15
3.4.- Actualizar	16
3.5.- Eliminar	17
<i>4.- Estructuras condicionales</i>	<i>19</i>

Desarrollo

1.- Polimorfismo

El polimorfismo es la característica de la programación orientada a objetos, que permite modificar la instancia de un objeto en tiempo de ejecución basado en una jerarquía de herencia. De esta forma, es posible generar una relación de vinculación denominada “binding”. El polimorfismo se puede realizar con clases superiores normales, abstractas e interfaces.

El objetivo del polimorfismo consiste en poder acceder a diferentes servicios en tiempo de ejecución sin necesidad de implementar diferentes referencias a objetos. Esta característica provee una gran flexibilidad en el proceso de desarrollo y ejecución de la aplicación.

En otras palabras, el polimorfismo es la capacidad que tienen los objetos de una clase en ofrecer respuestas distintas e independiente en función de los parámetros (diferentes implementaciones) utilizados durante su invocación. El objeto como entidad puede contener valores de diferentes tipos durante la ejecución del programa.

En Java el término polimorfismo también suele definirse como ‘Sobrecarga de parámetros’, lo cual en principio puede generar confusión. En realidad, suele confundirse con el tipo de polimorfismo más común, pero no es del todo exacto usar esta denominación.

1.1.- Ejemplo de Polimorfismo

Un ejemplo clásico de polimorfismo es el siguiente. Es posible crear dos clases distintas: Gato y Perro, que heredan de la superclase Animal. La clase Animal tiene el método generarSonido(), que se implementa de forma distinta en cada una de las subclases (gatos y perros claramente emiten sonidos diferentes).

Las clases antes mencionadas se verían de la siguiente manera:

```
class Animal {
```

```
public void generarSonido() {  
    System.out.println("Grr...");  
}  
}
```

```
class Gato extends Animal {  
    public void generarSonido() {  
        System.out.println("Miau");  
    }  
}
```

```
class Perro extends Animal {  
    public void generarSonido() {  
        System.out.println("Guau");  
    }  
}
```

Entonces, una cuarta clase puede enviar el mensaje de hacer sonido a un grupo de objetos Gato y Perro por medio de una variable de referencia de clase Animal, haciendo así un uso polimórfico de dichos objetos respecto del mensaje generarSonido.

```
public class Programa  
{  
    public static void main (String[] args) {  
        Animal spike = new Perro();  
        Animal benito = new Gato();  
  
        System.out.print("El gato hace: ");  
        benito.generarSonido();  
  
        System.out.print("El perro hace: ");  
        spike.generarSonido();  
    }  
}
```

Se puede apreciar en el último caso que se usa el método print en vez de println. La diferencia entre ambos es que el primero no agrega un salto de línea después de desplegar el texto indicado por consola. El método generarSonido(), en ambos casos, retorna un salto de línea al final del mensaje.

Finalmente, el resultado obtenido en la consola sería el siguiente:

```
El gato hace: Miau  
El perro hace: Guau
```

El término “polimorfismo” se refiere a la idea de "tener muchas formas", y ocurre cuando hay una jerarquía de clases relacionadas entre sí a través de la herencia. El caso anterior es un buen ejemplo de ello.

Por lo general existen 3 tipos de polimorfismo:

- **Sobrecarga:** Es el más conocido y se aplica cuando existen funciones con el mismo nombre en clases que son completamente independientes una de la otra.
- **Paramétrico:** Se produce cuando existen funciones con el mismo nombre, pero se usan diferentes parámetros (nombre o tipo). Se selecciona el método dependiendo del tipo de datos que se envíe.
- **Inclusión:** Es cuando se puede llamar a un método sin tener que conocer su tipo, así no se toma en cuenta los detalles de las clases especializadas, utilizando una interfaz común.

En líneas generales en lo que se refiere a la POO, la idea fundamental es proveer una funcionalidad predeterminada o común en la clase base y de las clases derivadas se espera que provean una funcionalidad más específica.

1.2.- Polimorfismo paramétrico

Antes se vio un ejemplo clásico de polimorfismo basado en sobrecarga. Ahora se hará un ejemplo Paramétrico. Es importante entender que la conversión automática sólo se aplica si no hay ninguna coincidencia directa entre un parámetro y argumento.

Aquí el método `demo()` se sobrecarga 3 veces: el primer método tiene 1 parámetro `int`, el segundo método tiene 2 parámetros `int` y el tercero tiene un parámetro `doble`. Por lo que para lidiar con esta variedad el método que se llamará está determinado por los argumentos que se entregan al llamar a los métodos. Esto sucede en tiempo de compilación, por lo que este tipo de polimorfismo se conoce también como polimorfismo en tiempo de compilación.


```
class Parametrico
{
    void demo (int a)
    {
        System.out.println ("a: " + a);
    }

    void demo (int a, int b)
    {
        System.out.println ("a y b: " + a + ", " + b);
    }

    double demo(double a) {
        System.out.println("double a: " + a);
        return a*a;
    }
}
```

Para ejecutar el caso anterior, se puede crear una clase con un método main(), en la cual se creará un objeto de la clase Parametrico y se invocará al método demo con diferentes parámetros.

```
class ProgramaParametrico
{
    public static void main (String args [])
    {
        Sobrecarga Obj = new Sobrecarga();
        double resultado;
        Obj.demo(10);
        Obj.demo(10, 20);
        resultado = Obj.demo(5.5);
        System.out.println("Tercer método: " + resultado);
    }
}
```

La salida de la clase anterior al invocar el método main() será la siguiente:

```
a: 10
a y b: 10,20
```

```
double a: 5.5  
Tercer método: 30.25
```

1.3.- Polimorfismo de inclusión

La habilidad para redefinir por completo el método de una superclase en una subclase es lo que se conoce como polimorfismo de inclusión (o redefinición).

En él, una subclase define un método que existe en una superclase con una lista de argumentos (si se define otra lista de argumentos, se estaría haciendo sobrecarga y no redefinición).

Un ejemplo muy básico en donde la clase Torre sobrescribe el método mover existente en su clase padre, llamada Pieza. Esto es el polimorfismo de inclusión.

```
abstract class Pieza{  
    public abstract void mover(int X, int Y);  
}
```

Una clase abstracta es aquella de la que no se pueden declarar instancias; dicho de otra manera, no se pueden declarar objetos de una clase abstracta. La finalidad de una clase abstracta es servir como clase base para otras clases a las que generalmente se conoce como clases "concretas".

La clase Torre heredará de la clase Pieza. En este caso, el comando `@Override` es necesario para sobrescribir el método antes definido.

```
class Torre extends Pieza{  
    @Override  
    public void mover(int X, int Y){  
        //acciones del método en la clase Pieza  
    }  
}
```

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

¿Cuál es la relación existente entre una interface o interfaz y una clase abstracta?



2.- Lectura de teclado

La clase `System.in` permite leer datos introducidos por el usuario desde teclado, pero su uso no es tan sencillo como el de la clase `System.out`.

Desde la versión 5,0, Java incluye una clase para realizar de forma sencilla la entrada de datos al programa. En los ejemplos siguientes se presenta cómo gestionar la entrada desde teclado usando dicha clase, llamada `Scanner`.

2.1.- La clase `Scanner`

La Clase `Scanner` forma parte del paquete `java.util` y permite leer valores introducidos por el teclado de una forma cómoda para el programador. Esta clase ofrece un grupo de métodos que bloquean el flujo de control del programa hasta que el usuario introduce una entrada por teclado. Además, permiten gestionar la entrada de teclado como si fuera un buffer de datos en el que se almacena lo que va tecleando el usuario para ser posteriormente leído.

```
import java.util.Scanner;

public class TestScanner {
    public static void main(String[] args) {
        String nombre;
        int a1, a2;
        Scanner teclado = new Scanner(System.in);
        System.out.println("Introduce tu nombre: ");
        nombre = teclado.nextLine();
        System.out.println("Introduce año de nacimiento y actual");
        a1 = teclado.nextInt();
        a2 = teclado.nextInt();
        System.out.print("Te llamas " + nombre);
        System.out.println(" y tienes " + (a2 - a1) + " años");
    }
}
```

El ejemplo anterior muestra un ejemplo de utilización de la clase Scanner para la lectura de valores por teclado. En primer lugar, se importa la clase Scanner del paquete correspondiente. A continuación, se crea un nuevo objeto de tipo Scanner pasándole como argumento el flujo de entrada estándar, System.in.

Dado que, por defecto, la entrada estándar del programa va ligada al teclado, esto permite leer los valores solicitados al usuario que este introduzca a través de teclado. Luego, se pide al usuario que introduzca su nombre. Es muy importante mostrar un mensaje al usuario antes de realizar cualquier operación de lectura de datos para que el usuario conozca que tipo de información debe suministrar al programa.

Mediante el método nextLine() se consigue leer desde el teclado una línea completa, es decir, todo lo que el usuario haya tecleado hasta que pulso la tecla Enter. Después, se solicita al usuario que introduzca tanto su año de nacimiento como el año actual. Ambos valores deben ser introducidos separados por, al menos, un espacio (un tabulador o un salto de línea) y son leídos mediante dos invocaciones consecutivas al método nextInt(). Finalmente, el programa muestra por pantalla tanto el nombre del usuario como su edad.

Al ejecutar el ejemplo anterior, un ejemplo del resultado obtenido al ingresar datos sería el siguiente:

```
Introduce tu nombre: Luisa García  
Introduce tu año de nacimiento y el actual 1987 2016  
Te llamas Luisa García y tienes 29 años
```

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

Si se desea rescatar desde el teclado un valor decimal ¿qué método podríamos usar?

Se necesita crear un programa que calcule el volumen de un cilindro dado su radio y su altura. ¿Se debería usar la clase Scanner? ¿Por qué?



3.- Administración de datos en colecciones

En la semana anterior se indicó el concepto de colecciones y se analizaron algunas operaciones elementales sobre ellas. El tipo más común de estas estructuras es la clase ArrayList, la cual se puede considerar como un arreglo que no tiene un tamaño fijo, y puede ser encontrado en el paquete java.util.

La diferencia entre un arreglo o matriz y un ArrayList en Java es que el tamaño de una matriz no se puede modificar (si desea agregar o eliminar elementos de una matriz, debe crear uno nuevo). Mientras que los elementos se pueden agregar y eliminar de un ArrayList cuando se desee. La sintaxis también es ligeramente diferente; en el ejemplo siguiente se indica cómo crear una colección de este tipo:

```
// importar la clase ArrayList
import java.util.ArrayList;

// Crea un objeto ArrayList
ArrayList<String> autos = new ArrayList<String>();
```

Tal como se aprecia en el ejemplo anterior, para usar los ArrayList se debe importar la clase antes de la declaración del código, en las primeras líneas. Además, en el ejemplo la lista podrá almacenar solo cadenas de texto.

3.1.- Agregar

La clase ArrayList tiene muchos métodos útiles. Por ejemplo, para agregar elementos a un ArrayList, se puede usar el método add():

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> autos = new ArrayList<String>();
        autos.add("Volvo");
        autos.add("BMW");
        autos.add("Ford");
        autos.add("Mazda");
        System.out.println(autos);
    }
}
```

En este ejemplo se crea un ArrayList con marcas de autos, y se agregan distintos valores uno a uno. Finalmente, se despliegan los datos existentes en el ArrayList por consola, tal como se muestra a continuación:

```
[Volvo, BMW, Ford, Mazda]
```

3.2.- Acceder a un ítem (listar)

Para acceder a un elemento en ArrayList, se puede usar el método `get()` y consultar un valor de acuerdo con el número de índice. Es importante recordar que los índices parten en 0, y terminan en $n-1$, donde n es la cantidad de elementos que el ArrayList tiene en un momento dado.

```
autos.get(0);
```

En el ejemplo anterior se obtiene el dato que está al principio de la lista. Si al momento de realizar esta acción la lista está vacía, este comando arrojará un error. En este caso, el resultado obtenido sería el siguiente:

```
Volvo
```

3.3.- Listar todos

Tal como se vio anteriormente, por medio del método `System.out.println` es posible mostrar todos los datos de un ArrayList, independiente de su tipo. Si bien cumple con el objetivo, el ideal sería poder acceder a cada elemento uno por uno. Esto solo se puede lograr por medio de un ciclo.

Un ciclo es una estructura dentro de la programación que permite hacer una acción o un conjunto de acciones un número determinado de veces, o bien hasta que se cumpla una condición.

Para recorrer los elementos de un ArrayList se puede un bucle o ciclo "for", y utilizar el método "`size()`" para especificar cuántas veces debe ejecutarse el bucle.

```
public class Main {  
    public static void main(String[] args) {  
        ArrayList<String> autos = new ArrayList<String>();  
        autos.add("Volvo");  
        autos.add("BMW");  
        autos.add("Ford");  
        autos.add("Mazda");  
  
        for (int i = 0; i < autos.size(); i++) {
```

```
        System.out.println(autos.get(i));
    }
}
```

También puede recorrer un ArrayList con el bucle for-each:

```
public class Main {
    public static void main(String[] args) {
        ArrayList<String> autos = new ArrayList<String>();
        autos.add("Volvo");
        autos.add("BMW");
        autos.add("Ford");
        autos.add("Mazda");

        for (String i : autos) {
            System.out.println(i);
        }
    }
}
```

3.4.- Actualizar

Para modificar un elemento del ArrayList se puede usar el método set(), entregando como parámetro el número de índice:

```
autos.set(0, "Opel");
```

Es necesario recordar que el primer parámetro del método set() es la posición en el ArrayList que se desea modificar, mientras que el segundo es el valor que se desea escribir en el espacio indicado.

Si se indica un valor que está fuera de rango, el programa arrojará un error. Si se listan los elementos después de la modificación se obtendrá el siguiente resultado:

```
Opel
BMW
Ford
```


Mazda

3.5.- Eliminar

Para eliminar un elemento del ArrayList, se puede utilizar el método `remove()`, indicando el número de índice:

```
autos.remove(0);
```

En el ejemplo anterior, se eliminará el primer elemento del listado. Esto implica que la cantidad de elementos de la lista disminuirá en una unidad.

Para eliminar todos los elementos de un ArrayList, se puede utilizar el método `clear()`:

```
autos.clear();
```

ANTES DE CONTINUAR CON LA LECTURA...REFLEXIONEMOS

Si se sabe que existe un método que permite obtener el largo de un ArrayList, ¿sería posible acceder al último valor de una colección de este tipo? ¿Por qué?



4.- Estructuras condicionales

Este tema se ha abordado en cursos anteriores, usando como referencia otros lenguajes de programación. Como ya se ha dicho, una estructura condicional permite decidir una acción en base a una condición. Para esto se usa en Java el comando “if”.

```
if (condicion){  
    //las acciones aquí descritas se harán solo si la condición  
    //se cumple  
}
```

El comando “if” puede ir acompañado de “else”; esta opción indicará las acciones a realizar en caso que la condición no se cumpla. La sintaxis sería la siguiente:

```
if (condicion){  
    //si la condición es verdadera  
}  
else{  
    //si la condición no se cumple  
}
```

Una condición puede ir dentro de otra. Por ejemplo, si se desea analizar los valores de un número. En este caso, las condiciones son excluyentes entre sí, vale decir, solo una de ellas se ejecutara en cualquier caso.

```
if (numero > 0) {  
    System.out.println("El número es mayor que cero");  
}  
else if (numero < 0){  
    System.out.println("El número es menor que cero");  
}  
else{  
    System.out.println("El número es igual a cero");  
}
```

Conclusión

Este ha sido el documento de la semana cuatro de la asignatura. En ella se ha visto el concepto de polimorfismo, y los tipos que existen en la programación orientada a objetos, en específico en el uso del lenguaje Java.

Además, se revisó el concepto de lectura de datos por teclado, usando la clase Scanner que es propia del mismo lenguaje analizado. Por último, se han visto los métodos que incluye la clase ArrayList para agregar elementos, listar de a uno o todos a la vez, eliminarlos y actualizarlos.

Al igual que en ocasiones anteriores, queda pendiente ver las cápsulas de la semana, tanto procedimental como conceptual, y realizar los ejercicios propuestos. Como siempre, se plantean las siguientes preguntas:

Referencias

[1] Flórez Fernández, H. A. (2012). Programación orientada a objetos usando java. Bogotá, Colombia: Ecoe Ediciones.

[2] Polimorfismo en Java: Programación orientada a objetos. IfgeekthenNTTdata. Referencia: <https://ifgeekthen.nttdata.com/es/polimorfismo-en-java-programación-orientada-objetos>

[3] Prieto Saez, N. y Casanova Faus, A. (2016). Empezar a programar usando Java (3a. ed.). Valencia, Spain: Editorial de la Universidad Politécnica de Valencia.

[4] Java ArrayList, W3 Schools. Referencia: https://www.w3schools.com/java/java_arraylist.asp

